

**SciPy 2025**

July 7 - July 13, 2025

Proceedings of the 24th
Python in Science Conference
ISSN: 2575-9752

A Lightweight Pipeline for Rewards-Guided Synthetic Text Generation Using NeMo and RAPIDS

Allison Ding¹  ¹NVIDIA Corporation

Abstract

Synthetic data generation (SDG) plays an increasingly important role in modern machine learning workflows. It addresses the dual challenges of data scarcity and privacy constraints, which often prevent the use of real-world datasets in sensitive domains such as healthcare, law, and finance. Although progress has been made in the structured data domain, especially with tools like SDV and CTGAN, the generation of high-quality synthetic text remains a challenging and underdeveloped area. Many existing SDG pipelines lack mechanisms for semantic control, offer limited quality assurance, and are not designed with computational efficiency in mind.

In this paper, we introduce a lightweight and modular SDG pipeline that is specifically designed for the generation of synthetic text data. Our approach integrates instruction-tuned large language models (LLMs) and reward-scoring models from NVIDIA's NeMo Curator with GPU-accelerated data processing tools from the RAPIDS ecosystem. The pipeline is implemented entirely within a Jupyter notebook, allowing for transparency, reproducibility, and ease of use. It includes stages for data cleaning, semantic deduplication, reward-guided generation, and iterative data augmentation. Throughout, the pipeline leverages RAPIDS libraries to accelerate operations such as clustering and filtering, making it suitable for large-scale experimentation. Empirical results from applying this pipeline to a legal QA dataset demonstrate improved data quality, reduced semantic redundancy, and efficient execution on GPU hardware. The solution is designed to be accessible to data scientists who may not specialize in LLM development, thereby filling a crucial usability gap in the SDG landscape.

Keywords Synthetic Data Generation, Reward Scoring, Semantic Deduplication, GPU Acceleration, NeMo, RAPIDS

1. INTRODUCTION

Synthetic data has become indispensable in machine learning, particularly in scenarios where real data is scarce, sensitive, or restricted by privacy regulations. The ability to generate realistic, high-quality synthetic samples enables model development, fine-tuning, and benchmarking without exposing sensitive content. In structured domains, tools such as SDV, CTGAN, and Gretel have emerged as practical solutions for generating tabular data that preserves statistical properties while mitigating privacy risks [1], [2]. However, the generation of unstructured or semi-structured data such as synthetic text remains substantially more complex. This complexity arises from the inherently semantic nature of language, which makes quality difficult to assess and even harder to enforce.

Text generation pipelines face several limitations that reduce their utility in practical workflows. First, many systems rely on large language models without sufficient post-generation validation, resulting in outputs that may be fluent but semantically irrelevant or redundant. Second, few pipelines offer robust mechanisms to filter out low-quality or factually incorrect samples, which is essential when synthetic data is meant to support downstream tasks such as supervised learning or knowledge extraction. Third, most available implementations are not optimized for interactive or scalable use, often lacking GPU acceleration or modularity for iteration and analysis.

Published Jul 10, 2025**Correspondence to**
Allison Ding
allisond@nvidia.com**Open Access** 

Copyright © 2025 Ding. This is an open-access article distributed under the terms of the [Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by-nc-nd/4.0/) license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

To address these gaps, we propose a pipeline that is reward-aware, reproducible, and optimized for real-world use. Built entirely in a Jupyter notebook, the pipeline connects NeMo’s instruction-tuned language models and reward scorers [3] with RAPIDS’ GPU-accelerated data handling and clustering libraries [4]. The design is intentional in its focus on usability, aiming to provide data scientists with a transparent and reproducible tool for generating and curating synthetic text data. The pipeline supports full-cycle generation workflows, from preprocessing and semantic deduplication to filtering and output, all with minimal setup and high computational efficiency.

2. BACKGROUND AND RELATED WORK

Structured data generation has seen considerable progress, with tools like SDV, CTGAN, and Gretel offering accessible APIs and statistical validation mechanisms [1], [2]. These systems have enabled synthetic data use in regulated environments such as healthcare and financial services. In contrast, synthetic text generation has largely been approached through either simple prompt-based generation using pretrained LLMs [5] or through more complex reinforcement learning pipelines, such as reinforcement learning with human feedback (RLHF). Both approaches have their strengths but also introduce new challenges in terms of complexity, reproducibility, and cost.

Prompt-based pipelines are typically easy to initiate but provide limited control over output quality. Without built-in validation or filtering, the generated text can be inconsistent, incoherent, or repetitive. On the other hand, RLHF frameworks introduce reward models that evaluate generated outputs, allowing quality-guided training or generation. However, implementing RLHF involves substantial engineering efforts, including reward function design, reinforcement learning infrastructure, and hyperparameter tuning, which make it impractical for many real-world cases.

Our approach seeks to fill the methodological gap between these extremes. Rather than training models through reinforcement learning, we integrate pretrained reward models directly into the generation pipeline [3]. This allows us to evaluate and filter synthetic samples after generation using a scalar quality score that encapsulates properties such as helpfulness, coherence, relevance, and alignment with instructions [6]. This mechanism introduces a layer of semantic quality control without the need for retraining or fine-tuning.

In addition, semantic deduplication is incorporated to ensure content diversity. This component addresses the issue of redundancy, which is especially important in iterative generation scenarios. Finally, the pipeline is built around RAPIDS, a suite of GPU-accelerated Python libraries that leverage the same APIs with the traditional pandas DataFrame and scikit-learn-based operations [4]. By integrating RAPIDS, we improve the scalability and responsiveness of the pipeline, requiring zero to minimal change in coding habits.

3. METHODOLOGY

3.1. Pipeline Overview

The synthetic text generation pipeline comprises six core components: data loading and cleaning, semantic deduplication, prompt-based generation, reward scoring, integration of accepted samples, and GPU acceleration. Each step is designed to be modular and independently testable. The entire pipeline is orchestrated within a single notebook and supports interactive development, reproducibility, and iteration. These properties are essential for fast experimentation, especially when working with domain-specific data such as legal documents or biomedical QA pairs, where high quality, minimal redundancy, and efficiency in execution are critical.

3.2. Data Loading and Cleaning

The initial stage involves loading raw data, typically stored in JSONL format. The dataset we use as an example contains question-answer pairs and optional metadata such as titles, tags, or confidence scores. The cleaning process removes HTML tags, corrects malformed Unicode characters, and applies filters based on word count and quality scores. For example, questions and answers are retained only if their lengths

fall within acceptable ranges and their associated scores exceed a defined threshold. This step ensures that only relevant and well-formed records are passed into subsequent stages.

```
# Data Loading

# Import data
import pandas as pd

path = './peft-curation-with-sdg-70b/data/raw/splits/law-qa-train.jsonl'

# Load your initial dataset
dataset = pd.read_json(
    './peft-curation-with-sdg-70b/data/raw/splits/law-qa-train.jsonl', lines = True
)
```

```
# Data Cleaning Functions

from bs4 import BeautifulSoup
import re

def clean_html(text):
    if not isinstance(text, str):
        return ""
    text = BeautifulSoup(text, "lxml").get_text()
    return re.sub(r"\s+", " ", text).strip()

import ftfy
from ftfy import TextFixerConfig

fix_config = TextFixerConfig()

def fix_unicode(text):
    if not isinstance(text, str):
        return ""
    return ftfy.fix_text(text, config=fix_config)

def word_count(text, min_words=50, max_words=500):
    words = text.strip().split()
    return min_words <= len(words) <= max_words

def filter_low_score(score, threshold=0):
    return score >= threshold
```

```
# Data Cleaning Pipeline

def clean_filter_by_row(row):
    # Clean + fix each text field
    for field in ["title", "question", "answer"]:
        text = clean_html(row[field])
        text = fix_unicode(text)
        row[field] = text

    # Apply filters
    question_bool = word_count(row["question"]) and filter_low_score(float(row["question_score"]))
    answer_bool = word_count(row["answer"]) and filter_low_score(float(row["answer_score"]))

    return question_bool and answer_bool

def data_clean(dataset):
    dataset["keep"] = dataset.apply(clean_filter_by_row, axis=1)
    clean_dataset = dataset[dataset["keep"]].drop(columns=["keep"]).reset_index(drop=True)
    dataset.drop("keep", axis = 1, inplace = True)

    return clean_dataset
```

3.3. Semantic Deduplication

To avoid training or evaluating models on semantically repetitive content, we implement a deduplication step based on sentence embeddings. Each record is converted into a single textual representation by

concatenating the title, question, and answer fields. Sentence embeddings are computed using a pretrained SentenceTransformer model and normalized using CuPy to enable efficient GPU operations. KMeans clustering is then applied in the embedding space to identify groups of similar samples. Within each cluster, cosine similarity is used to identify redundant records. Those with high proximity to the cluster centroid are marked for removal. This process significantly improves content diversity and reduces the likelihood of overfitting in downstream tasks.

```
# Semantic Dedupe

def semantic_dedupe(dataset):
    dataset["text"] = dataset["title"] + dataset["question"] + dataset["answer"]

    # Step 1: Generate normalized sentence embeddings
    from sentence_transformers import SentenceTransformer
    import cupy as cp

    model = SentenceTransformer("all-MiniLM-L6-v2")
    embeddings = model.encode(dataset["text"].tolist(), batch_size = 128, show_progress_bar = True,
    convert_to_numpy = True)
    embeddings_norm = cp.array(embeddings)/cp.linalg.norm(cp.array(embeddings), axis = 1, keepdims =
    True)

    # Step 2: Perform KMeans with cosine normalization (Euclidean KMeans on normalized vectors)
    from sklearn.cluster import KMeans
    from sklearn.metrics.pairwise import cosine_similarity

    n_clusters = 1000
    kmeans = KMeans(n_clusters = n_clusters, random_state = 1234, max_iter = 100)
    labels = kmeans.fit_predict(embeddings_norm)
    centroids = kmeans.cluster_centers_

    # Step 3: Deduplication logic (find points within eps_to_extract)
    eps_to_extract = 0.01
    dedup_indices = []
    for cluster_id in range(n_clusters):
        cluster_points = cp.where(cp.array(labels) == cluster_id)[0]
        if len(cluster_points) == 0:
            continue

        cluster_embeddings = embeddings_norm[cluster_points].get()
        centroid = centroids[cluster_id].reshape(1, -1)
        sims = cosine_similarity(cluster_embeddings, centroid).flatten()
        within_eps = cluster_points[sims >= (1 - eps_to_extract)]

        if len(within_eps) > 0:
            dedup_indices.extend(within_eps)

    # Step 4: Get deduplicated data
    mask = dataset.index.isin(dedup_indices)
    kept_data = dataset[~mask]

    # step 5: drop the added column
    dataset.drop("text", axis = 1, inplace = True)
    kept_data.drop("text", axis = 1, inplace = True)

    return kept_data
```

3.4. Synthetic Data Generation with Rewards

Synthetic text generation is performed using NeMo's Nemotron-70B model, which is instruction-tuned to support tasks such as question generation and paraphrasing. Prompt templates are designed for three specific functions: generating questions from answers, paraphrasing questions, and paraphrasing answers. For each function, the generator outputs a candidate sample that is then evaluated by a pretrained reward model. This model, also available in the NeMo ecosystem, assigns a scalar score reflecting the overall quality of the text. Only those samples with scores falling within a specified range are retained. This scoring function enables automated quality control and reduces reliance on manual inspection or external validation.

```

# Synthetic Data Generation

def SDG(dataset, sample_size_perc = 0.001, reward_threshold = 0):

    from openai import OpenAI
    from nemo_curator import OpenAIClient
    from nemo_curator.synthetic import NemotronGenerator
    import random

    openai_client = OpenAI(
        base_url="https://integrate.api.nvidia.com/v1",
        api_key=""
    )
    client = OpenAIClient(openai_client)
    generator = NemotronGenerator(client)

    n_variants = 1
    sdg_model = "nvdev/nvidia/llama-3.1-nemotron-70b-instruct"
    sdg_model_kwargs = {
        "temperature": 0.2,
        "top_p": 0.7,
        "max_tokens": 1024,
        "seed": 1234
    }

    reward_model = "nvdev/nvidia/llama-3.1-nemotron-70b-reward"

    PROMPT_GENERATE_QUESTIONS_FROM_ANSWER = """TEXT:
{document}

Given the above text, generate exactly {n_openlines} questions that can be answered by the text.
All questions must be answerable by the text and be relevant to the text.
Do not directly reference the text in the questions.
Every question should be a complete sentence and end with a question mark. There should be no
other text besides the questions.
Begin each question with `* ` and end each question with a newline character. Also, each question
must be concise.
Make sure to generate exactly {n_openlines} questions.
"""

    PROMPT_PARAPHRASE_TEXT = """TEXT:
{document}

Given the above text, paraphrase the text. Produce exactly {n_openlines} variants.
There should be no other text besides the paraphrased text.
The paraphrased text must be shorter than the original text. The paraphrased text must be
factually correct and relevant to the original text.
Begin each variant with `* ` and end each variant with a newline character.
Make sure to generate exactly {n_openlines} variants.
"""

    output = []

    N = len(dataset)
    n = int(len(dataset) * sample_size_perc)
    sample_indices = random.sample(range(N), n)

    for idx in sample_indices:
        row = dataset.iloc[idx]
        question = row["question"]
        answer = row["answer"]

        gen_title = generator.generate_closed_qa_instructions(
            document=answer,
            n_openlines=n_variants,
            prompt_template=PROMPT_GENERATE_QUESTIONS_FROM_ANSWER,
            model=sdg_model,
            model_kwargs=sdg_model_kwargs,
        )

        gen_question = generator.generate_closed_qa_instructions(
            document=question,
            n_openlines=n_variants,
            prompt_template=PROMPT_PARAPHRASE_TEXT,

```

```

        model=sdg_model,
        model_kwargs=sdg_model_kwargs,
    )

    gen_answer = generator.generate_closed_qa_instructions(
        document=answer,
        n_openlines=n_variants,
        prompt_template=PROMPT_PARAPHRASE_TEXT,
        model=sdg_model,
        model_kwargs=sdg_model_kwargs,
    )

    messages = [
        {"role": "user", "content": f"{gen_title[0]}\n\n{gen_question[0]}"},
        {
            "role": "assistant",
            "content": f"{gen_answer[0]}",
        },
    ]

    rewards = client.query_reward_model(messages=messages, model=reward_model)

    if rewards >= reward_threshold:
        row_new = {
            "id": f"{row['id']}-synth-{n_variants}",
            "question": f"{gen_question[0]}",
            "answer": f"{gen_answer[0]}",
            "title": f"{gen_title[0]}",
            "file_name": "law-stackexchange-questions-answers.json.synth",
            "tags": f"{row['tags'] * n_variants}",
            "question_score": f"{rewards}",
            "answer_score": f"{rewards}"
        }

        output.append(row_new)

    output_df = pd.DataFrame(output)

    return output_df

```

3.5. Complete SDG Pipeline

The complete pipeline integrates all components into an iterative loop. The process begins with cleaning and deduplicating the original dataset to establish a high-quality baseline. In each subsequent iteration, new synthetic samples are generated and merged with the deduplicated dataset from the previous round. The combined dataset is then subjected to semantic deduplication to eliminate newly introduced redundancy. This iterative procedure supports the progressive enrichment of the dataset while maintaining high semantic diversity and quality standards. All outputs, including intermediate generations and reward scores, are retained to facilitate auditability and further analysis.

```
# Synthetic Data Generation Pipeline

def SDG_Rewards_Pipeline(dataset, round_num, sample_size_perc = 0.001, reward_threshold = 0):

    cleaned_dataset = data_clean(dataset)
    deduped_dataset = semantic_dedupe(cleaned_dataset)
    print(f"After the initial curation, the dataset has {len(deduped_dataset)} records (originally {len(dataset)}).")

    dataset = deduped_dataset
    for num in range(round_num):

        sdg_dataset = SDG(dataset, sample_size_perc = sample_size_perc, reward_threshold = reward_threshold)
        dataset = pd.concat([dataset, sdg_dataset], axis = 0)
        deduped_dataset = semantic_dedupe(cleaned_dataset)

        print(f"After round {num + 1}, the dataset has {len(deduped_dataset)} records (originally {len(dataset)}).")

    return deduped_dataset
```

3.6. GPU Acceleration

To support high-throughput execution, the pipeline leverages GPU acceleration for key operations. DataFrame manipulations, embedding normalization, clustering, and scoring are performed using RAPIDS libraries such as cuDF and cuML [4]. Users can activate acceleration by loading RAPIDS extensions at the top of the notebook, without altering their existing data manipulation code. This drop-in acceleration enables the pipeline to process tens of thousands of records interactively, making it practical for both prototyping and production.

```
# Activate GPU Acceleration

%load_ext cuml.accel
%load_ext cudf.pandas
```

4. EXPERIMENTS AND RESULTS

4.1. Use Case Setup

To evaluate the performance and effectiveness of the proposed pipeline, we applied it to a real-world dataset containing 19,474 text records. This dataset served as the basis for a multi-stage process involving semantic deduplication, prompt-based generation, reward scoring, integration of accepted samples. The evaluation was conducted using a single GPU-enabled machine to validate the feasibility of interactive, large-scale processing within a notebook environment.

An initial baseline was established by cleaning and deduplicating the original dataset. This step reduced the corpus to 12,244 records, removing approximately 37.2% of entries that (1) contained fewer than 50 or more than 500 words, (2) failed basic HTML cleaning or Unicode normalization, or (3) were semantically redundant based on sentence embeddings and cosine similarity filtering. This baseline served as the foundation for the subsequent synthetic data generation stage.

4.2. Quantitative Evaluation

The pipeline was configured to run for 10 rounds, each involving the generation of synthetic samples sampled at a rate of 0.1% from the current dataset. A reward threshold of -20 was applied to filter outputs based on quality, using scalar reward scores to retain only fluent, relevant, and contextually appropriate text.

As shown in the table, each round resulted in a modest increase in the dataset size before deduplication. For example, after round 1, the dataset increased from 12,244 to 12,249 records. This pattern continued across

Round	Original Records (Pre-Dedup)	Records After Dedup
Initial	19,474	12,244
1	12,249	12,244
2	12,253	12,244
3	12,258	12,244
4	12,263	12,244
5	12,265	12,244
6	12,268	12,244
7	12,271	12,244
8	12,275	12,244
9	12,281	12,244
10	12,285	12,244

all 10 iterations, with post-generation counts ranging from 12,249 to 12,285. After each merge, semantic deduplication was applied, bringing the dataset size back to 12,244, confirming that only non-redundant and unique samples were retained.

This process illustrates the effectiveness of the deduplication step. Across all 10 rounds, redundant content introduced during generation was systematically identified and removed. Despite generating over 400 synthetic candidates in accumulation, the dataset size remained stable, reflecting strict enforcement of semantic quality constraints.

4.3. Visual Outputs

Reward score histograms further supported the filtering logic by revealing clustering around acceptable reward thresholds and removal of semantically duplicated phrasing. These visualizations confirm that the semantic filter is not only size-preserving but meaningfully content-selective.

5. CONCLUSIONS

We have presented a reproducible and scalable pipeline for reward-guided synthetic text generation. The system integrates instruction-tuned generation models with semantic deduplication and quality scoring, all within a GPU-accelerated environment. By relying on pretrained components and avoiding the complexity of reinforcement learning, the pipeline remains accessible to data scientists without deep expertise in LLM training. Its modular structure supports iterative enrichment of datasets while preserving semantic diversity and ensuring quality. All components are implemented in a Jupyter notebook, supporting transparency, auditability, and ease of extension. This work contributes a practical and scientifically grounded approach to responsible synthetic text generation in real-world machine learning settings.

REFERENCES

- [1] N. Patki, R. Wedge, and K. Veeramachaneni, "The synthetic data vault," in *2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2016, pp. 399–410. doi: [10.1109/DSAA.2016.49](https://doi.org/10.1109/DSAA.2016.49).
- [2] SDV Developers, "SDV: Synthetic Data Vault." [Online]. Available: <https://sdv.dev/>
- [3] NVIDIA, "NeMo Curator and Nemotron Models." [Online]. Available: <https://developer.nvidia.com/nemo>
- [4] RAPIDS AI, "cuDF and cuML: GPU-accelerated DataFrames and Machine Learning." [Online]. Available: <https://rapids.ai/>
- [5] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45. doi: [10.18653/v1/2020.emnlp-demos.6](https://doi.org/10.18653/v1/2020.emnlp-demos.6).
- [6] H. Gadre, S. Shen, W. L. Hamilton, and C. Raffel, "RewardBench: Benchmarking Alignment Rewards for Language Models," *arXiv preprint arXiv:2410.01257*, 2024, doi: [10.57967/hf/2457](https://doi.org/10.57967/hf/2457).